

RAG 面试题总结

RAG 是 AI 应用开发里最容易被低估的模块。

很多人以为 RAG 就是“文档切块 -> 转向量 -> 存向量库 -> 检索 -> 拼 Prompt”。Demo 阶段这么理解没问题，但一到真实业务，问题马上变复杂：文档解析不干净、Chunk 切碎了语义、Embedding 模型选错、召回结果不准、权限过滤漏了、知识库更新后旧版本还在、模型拿到证据却没有正确回答。

所以，RAG 面试真正考的不是“你会不会接向量数据库”，而是：**你能不能把一个检索增强生成系统拆成可定位、可优化、可评测、可更新的工程链路。**

这份 RAG 面试题根据 AI 专栏现有文章整理。建议你用下面这条主线复习：

1. 先理解 RAG 解决什么问题，以及它和微调、长上下文、传统搜索的区别。
2. 再理解 Embedding、相似度、ANN 索引和向量数据库选型。
3. 接着理解文档处理、Chunk 策略、元数据和权限过滤。
4. 然后掌握 Hybrid Search、Query Rewrite、Rerank、上下文压缩等优化手段。
5. 最后补上 GraphRAG、知识库更新和评测闭环。

1 面试官真正想考什么

RAG 题通常会从概念开始，但很快会追到排查和优化。你可以按下面几个层次准备。

考察方向	面试官想确认什么	常见扣分点
RAG 基础	你是否知道 RAG 解决知识更新、私有数据和可溯源问题	只说“降低幻觉”，讲不出链路
Embedding 和索引	你是否理解向量检索的近似性和成本取舍	把向量数据库当普通数据库
文档处理	你是否知道召回质量从文档进入系统前就开始决定	只调 TopK，不看解析和 Chunk
检索优化	你是否能定位召回不准、排序不准、上下文噪声问题	遇到效果差只改 Prompt
GraphRAG	你是否理解多跳关系和全局问题为什么难	认为 GraphRAG 一定比向量 RAG 好
更新与评测	你是否能维护长期运行的知识库	没有版本、灰度、回滚和评测意识

回答 RAG 题时，尽量把问题拆成“数据进入索引前、检索召回时、上下文注入时、模型生成后、线上持续更新”几个阶段。这样面试官会更容易感受到你的系统化思维。

2 RAG 基础

参考文章：《万字详解 RAG 基础概念》

这一组题是 RAG 面试的入口。重点要讲清楚 RAG 的价值和边界：它不是让模型突然变聪明，而是给模型提供外部证据，让回答更可引用、可审计、可更新。

建议掌握这些关键点：

- RAG 主要解决大模型知识过时、缺少私有数据、回答不可溯源等问题。
- 传统搜索返回文档列表，RAG 返回基于证据综合后的答案。
- RAG 和微调不是替代关系。知识频繁变化、需要引用来源时优先 RAG；要固定风格、格式或能力倾向时再考虑微调。
- 长上下文适合少量材料深度分析，但企业级知识库仍然需要检索来控制成本、权限和噪声。
- RAG 不能彻底消灭幻觉。检索错、证据不足、上下文噪声、模型不遵循证据，都会导致错误答案。

高频面试题：

- 什么是 RAG？为什么需要 RAG？
- RAG 和传统搜索引擎有什么区别？
- RAG 和微调怎么选？什么时候用 RAG，什么时候微调，什么时候两者结合？
- RAG 系统中 Embedding 模型怎么选？为什么？

- 余弦相似度、内积和欧氏距离有什么区别？
- RAG 的幻觉问题怎么解决？RAG 一定不会产生幻觉吗？
- 什么是 Lost in the Middle 问题？如何应对？
- 长上下文窗口是否会取代 RAG？
- RAG 系统的评估指标有哪些？
- RAG 的优势和局限性是什么？
- 什么场景适合用 RAG？什么场景不适合？

一个更完整的回答方式是：RAG 的价值在于把模型回答绑定到可检索证据上，但它的上限由检索质量决定。如果正确证据没有被召回，后面的 Prompt 写得再漂亮也救不回来。

3 向量数据库与索引

参考文章：《万字详解 RAG 向量索引算法和向量数据库》

这一组题会考到一些底层概念，但面试官通常不是让你推公式，而是看你是否理解向量检索的取舍：速度、召回率、内存、构建成本、过滤能力和运维复杂度。

建议掌握这些关键点：

- Embedding 把文本映射到语义向量空间，相似文本在空间中距离更近。
- ANN 近似检索牺牲一部分精确性，换取更高查询性能，这是大规模向量检索的常见取舍。
- Flat 适合小规模和评测基准，HNSW 查询快但内存成本高，IVFFLAT 更节省资源但依赖聚类 and 参数调优。
- PostgreSQL + pgvector 适合中小规模和已有 PostgreSQL 技术栈，专业向量数据库更适合大规模、高并发、复杂检索场景。
- 向量检索经常要和元数据过滤、权限过滤、关键词检索结合，不能只看相似度。

高频面试题：

- 什么是 Embedding？为什么需要把文本转成向量？
- RAG 场景为什么需要向量数据库？
- ANN 算法为什么可以接受不是 100% 精确的结果？
- 有哪些向量索引算法？各自优缺点是什么？
- Flat、HNSW、IVFFLAT、IVF-PQ 分别适合什么场景？
- HNSW 和 IVFFLAT 有什么区别？
- HNSW 的 `ef_search` 参数怎么调？调大和调小分别会怎样？
- 向量数据库和传统数据库最核心的区别是什么？
- 如果向量数据从 100 万增长到 1 亿，架构上需要做什么调整？
- 为什么选择 PostgreSQL + pgvector？什么时候应该换专业向量数据库？

如果被问“向量数据库怎么选”，不要只报产品名。更好的回答是先问规模、延迟、过滤条件、运维能力、云服务偏好、数据安全要求，再给方案。技术选型不是榜单投票，而是约束匹配。

4 文档处理与 Chunk 策略

参考文章：《RAG 文档处理与切分策略：从解析、清洗、Chunking 到多模态内容处理》

很多 RAG 问题的根源不在模型，也不在向量库，而在文档处理。垃圾内容进索引，后面检索出来的也只是高相似度垃圾。

建议掌握这些关键点：

- 文档处理管线通常包括解析、清洗、结构化、切分、元数据补全、Embedding、入库和校验。
- Chunk 切分不能只按固定长度切。标题层级、段落语义、表格、代码块、FAQ、章节边界都要考虑。
- Chunk 太大，召回不精准且上下文成本高；Chunk 太小，语义不完整，容易丢失上下文。
- Overlap 可以缓解切分边界问题，但过大容易引入重复内容和检索噪声。
- 元数据很关键，包括来源、标题、页码、更新时间、权限范围、文档版本和业务标签。

高频面试题：

- RAG 文档处理管线通常包含哪些步骤？
- 文档解析、清洗、结构化分别解决什么问题？
- Chunk 切分为什么不能只按固定长度切？
- Chunk 大小、Overlap、语义边界应该怎么取舍？
- 表格、代码块、图片、多模态内容进入 RAG 前怎么处理？
- 文档处理阶段如何保留标题层级、页码、来源和权限元数据？
- Chunk 质量差会带来哪些召回和生成问题？
- 如何从零搭建一套企业级文档处理管线？

面试里如果问“Chunk 怎么切”，建议不要直接说固定 500 字或 1000 字。更稳的回答是：先根据文档类型和问答粒度确定基本范围；优先按标题、段落、语义边界切；对表格、代码、FAQ 做特殊处理；保留父级标题和元数据；最后通过检索评测验证 Chunk 策略，而不是凭感觉调参数。

5 RAG 检索优化

参考文章：《万字详解 RAG 优化：从召回、重排到上下文工程的系统调优》

这一组题最能体现实战经验。RAG 效果差时，不要一上来就改 Prompt。先判断问题发生在哪一段：没有召回正确证据、召回了但排得太后、放进上下文的内容太吵、模型没有正确使用证据，还是评测样本不稳定。

建议掌握这些关键点：

- Hybrid Search 结合关键词检索和向量检索，适合专业术语、编号、实体名、语义表达混杂的场景。
- Query Rewrite 解决用户问题表达不规范、口语化、多意图、缩写和上下文省略问题。
- Rerank 负责在候选结果里重新排序，解决向量相似度不等于答案相关性的问题。
- 上下文压缩可以降低噪声和成本，但压缩错误会丢失关键证据。
- RAG 优化必须基于失败样本集，不能只拿几条主观案例反复调。

高频面试题：

- RAG 召回率低应该怎么排查？
- Chunk 策略、Metadata、Hybrid Search、Query Rewrite、Rerank 分别解决什么问题？
- Hybrid Search 是什么？BM25 和向量检索怎么融合？
- Query Rewrite、HyDE、Self-Query 分别适合什么场景？
- Rerank 解决什么问题？为什么不能只依赖向量相似度排序？
- 上下文压缩有什么价值？什么时候会伤害答案质量？
- RAG 优化为什么必须先建立失败样本集？
- 线上 RAG 出现“答非所问”，应该按什么路径定位？

推荐的排查顺序是：先看正确文档是否进入候选池，再看排序位置是否靠前，再看上下文是否被截断或污染，最后看模型是否忠实使用证据。这样能避免把检索问题误判成 Prompt 问题。

6 GraphRAG

参考文章：《万字详解 GraphRAG：为什么只靠向量检索撑不起复杂知识问答》

GraphRAG 题通常出现在更深入的面试里。它不是标准 RAG 的银弹，而是用图结构补足向量检索在实体关系、多跳推理和全局性问题上的短板。

建议掌握这些关键点：

- 标准向量 RAG 擅长局部相似内容召回，但不擅长跨文档关系、多跳推理和全局总结。
- GraphRAG 会抽取实体和关系，构建知识图谱，再通过局部检索、全局检索或社区摘要回答复杂问题。
- 社区摘要可以帮助回答全局问题，但构建和更新成本很高，也可能引入摘要偏差。
- GraphRAG 的权限过滤比文档级过滤更复杂，因为节点、边、邻居和摘要都可能带来信息泄露。
- 成熟系统往往不是纯 GraphRAG，而是根据问题类型在关键词检索、向量检索、多向量、图检索之间动态路由。

高频面试题：

- GraphRAG 解决什么问题？和标准向量 RAG 有什么区别？
- 为什么说 Chunk 是信息孤岛？
- 向量相似度为什么不擅长多跳推理？
- GraphRAG 中实体、关系、社区发现分别是什么？
- 全局检索和局部检索有什么区别？
- GraphRAG 的社区摘要有什么价值？它的成本在哪里？
- GraphRAG 如何做权限过滤？
- 什么场景适合 GraphRAG？什么场景不适合？
- 成熟系统为什么通常不是纯 GraphRAG，而是混合路由架构？

如果被问“要不要上 GraphRAG”，不要默认回答要。更稳的判断是：如果业务问题大量涉及跨文档关系、组织网络、实体关联、多跳推理和全局总结，可以评估 GraphRAG；如果只是 FAQ、产品文档、政策查询，标准 RAG 加检索优化通常更划算。

7 知识库更新与评测

参考文章：《RAG 知识库文档如何更新：增量更新、版本控制、去重与全量重建》、《AI 应用评测体系：从 Golden Set 构建到线上灰度闭环》

RAG 上生产后，最容易被忽视的是“长期维护”。文档会更新，Embedding 模型会升级，Chunk 策略会调整，权限会变化，业务问题分布也会变。没有更新和评测机制，RAG 很快就会从“知识库问答”变成“旧知识随机复读”。建议掌握这些关键点：

- 知识库更新要处理新增、修改、删除、版本、去重、权限、灰度和回滚。
- Embedding 模型升级通常意味着向量空间变化，旧向量和向量混用会带来检索质量问题。
- Chunk 策略变更可能影响所有历史切片，通常需要全量重建。
- RAG 评测要分检索指标和生成指标。检索差和生成差，优化方向完全不同。
- 线上失败样本要回流到评测集，形成持续改进闭环。

高频面试题：

- RAG 知识库为什么不能只新增不删除？
- 增量更新和全量重建怎么选？
- Embedding 模型升级后，为什么通常需要重建索引？
- Chunk 策略变更会影响哪些历史数据？
- 如何避免同一文档多个版本同时被召回？
- 知识库更新如何做灰度、回滚和审计？
- RAG 评测为什么要分检索质量和生成质量？
- MRR、NDCG、Recall@K、Context Precision、Faithfulness 分别衡量什么？

回答更新题时，可以用“数据版本 + 索引版本 + 灰度发布 + 指标监控 + 快速回滚”这条线。这样比只说“定时同步文档”更像生产系统。

8 排查框架

RAG 效果差，可以按下面路径排查：

1. 问题理解：用户问题是否口语化、缩写、多意图、需要多跳推理。
2. 文档处理：原始文档是否解析正确，Chunk 是否保留语义和元数据。
3. 召回阶段：正确证据是否进入候选池，召回池是否足够大。
4. 排序阶段：正确证据是否排在前面，是否需要 Rerank。
5. 上下文阶段：证据是否被截断、重复、污染，是否存在 Lost in the Middle。
6. 生成阶段：模型是否忠实基于证据回答，是否需要引用和拒答策略。
7. 评测阶段：是否有稳定样本集，是否能复现问题。

这个框架非常适合面试，因为它能把 RAG 从“一个链路”拆成“多个可诊断模块”。

9 常见扣分点

- 把 RAG 简化成向量数据库接入。
- 只关注 TopK，不关注文档解析、Chunk、元数据和权限。
- 效果差时只改 Prompt，不看检索和排序。
- 认为向量相似度高就等于答案相关。
- 认为 GraphRAG 一定优于标准 RAG，不考虑成本和适用场景。
- 没有知识库版本管理、灰度、回滚和评测闭环。

10 复习建议

建议按“基础概念 -> 向量索引 -> 文档处理 -> 检索优化 -> GraphRAG -> 更新与评测”的顺序复习。

复习时要始终记住一句话：**RAG 的核心能力不是生成，而是把正确证据稳定、低成本、可治理地送到模型面前。**如果你能围绕这句话展开，RAG 面试基本不会跑偏。